WOXSEN UNIVERSITY

SCHOOL OF TECHNOLOGY

— ACADEMIC COURSE PROJECT REPORT —

Student & College Management System

Relational Database Modeling, High-Performance Java REST Engine, & Responsive React Web Dashboard

Candidate Name:	Atharva Kanojia
University Roll Number:	25WU0101019
Program & Section:	B.Tech Computer Science & Engineering (AIML Panthers)
Course Name	Database Management Systems
Faculty / Course Instructor:	Dr. Kiran Mayee Adavala
Project Repository Link:	https://github.com/kanojiaatharva/DBMS-Course-Project
Author GitHub Profile:	https://github.com/kanojiaatharva

2. Abstract

Modern higher educational institutions operate within complex academic ecosystems characterized by multifaceted interdependencies among institutional campuses, specialized academic departments, faculty researchers, dynamic course catalogs, and extensive student bodies. Traditional manual record-keeping methods and isolated, fragmented spreadsheet systems frequently suffer from critical limitations, including pervasive data redundancy, update and deletion anomalies, loss of referential integrity, and inadequate transactional safeguards. This project presents the comprehensive design, rigorous relational modeling, and full-stack software implementation of the Student and College Management System, an enterprise-grade academic information management platform.

The system is architected around a normalized relational database schema engineered on the MySQL 8.0+ Relational Database Management System (RDBMS), utilizing the transactional InnoDB storage engine. The schema satisfies Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF), establishing strict declarative constraints comprising primary keys, composite keys, foreign keys with explicit cascade and restriction policies, unique indexes, and domain check constraints. Data persistence is coupled with a high-performance, lightweight Java 21+ backend service engineered directly upon the JDK's embedded HttpServer and raw Java Database Connectivity (JDBC) via mysql-connector-j and Google Gson. By eliminating heavy application framework overhead, the backend achieves ultra-low latency execution (<500ms startup, sub-millisecond query dispatch).

The presentation tier is realized as a single-page web dashboard developed with React 18 and Vite. Rejecting generic UI templates, the interface adopts an editorial, developer-centric aesthetic powered by the JetBrains Mono monospace typeface, a warm parchment substrate, live dynamic badge counters, schema-driven drawer forms, two-step deletion confirmation safeguards, and real-time multi-column search filtering. Low-level database exceptions (e.g. MySQL error codes 1062, 1451, 1452, 3819) are transparently intercepted and translated into user-friendly actionable feedback, delivering an end-to-end ACID-compliant solution for collegiate academic administration.

3. Introduction and Problem Statement

3.1 Context & Background

Colleges and universities handle enormous volumes of operational and pedagogical data on a daily basis. An institution is not merely a flat list of students; it is a structured hierarchy comprising constituent colleges, specialized academic departments, instructional faculties, course curricula, advisor-advisee relationships, faculty teaching allocations, and multi-semester student course enrollments. In an era where institutional audits, continuous accreditation standards, and real-time student performance monitoring are paramount, maintaining reliable and immediately accessible academic records is essential to institutional operations.

3.2 Problem Statement

In many institutional settings, departments and faculty administrators still depend on disparate spreadsheets (e.g., Microsoft Excel, Google Sheets) or siloed legacy desktop software. These decoupled approaches exhibit severe operational deficiencies:

- **Data Redundancy & Update Anomalies:** Spreadsheet records duplicate entity attributes across multiple files, causing massive storage waste and synchronization failures when records are updated.
- **Loss of Referential Integrity:** Spreadsheets lack declarative relational enforcement. Deleting a department or course often leaves orphaned student enrollment or faculty records with dangling references.
- **Absence of Domain Validation:** Without database-level CHECK constraints or centralized domain typing, invalid credits, misspelled semester tags, and malformed letter grades easily corrupt data.
- **Lack of ACID Guarantees:** Flat files provide zero transactional concurrency control, leading to race conditions, lost updates, and dirty reads during simultaneous multi-user access.
- **Inefficient Data Retrieval:** Generating multi-entity administrative reports (such as a student's complete academic standing, advisor affiliations, and course history) requires cumbersome manual cross-referencing.

To decisively resolve these vulnerabilities, there is a compelling need for a robust, centralized Relational Database Management System (RDBMS) paired with an intuitive, tamper-proof user interface that enforces data integrity at the database layer.

4. Objectives and Scope

4.1 Project Objectives

- **Rigorous Conceptual & Logical Modeling:** Model the collegiate operational environment into an optimal Entity-Relationship (ER) model encompassing colleges, departments, faculty, courses, students, teaching assignments, and enrollments.
- **Full Normalization to 3NF/BCNF:** Decompose the relational tables to Third Normal Form (3NF) to eliminate insertion, deletion, and modification anomalies while preserving functional dependencies.
- **Integrity Constraint Enforcement:** Enforce primary keys, foreign key constraints (RESTRICT, CASCADE, SET NULL), unique constraints, and check constraints (valid grade scales and 1-6 credit ranges).
- **Lightweight High-Throughput Backend:** Develop a zero-overhead Java 21+ REST service utilizing direct JDBC and JDK HttpServer for ultra-fast query execution and custom database error translation.
- **Custom Developer-Grade Web Interface:** Build a responsive React 18 single-page dashboard using the JetBrains Mono typeface, schema-driven drawer modals, and live search filtering.
- **Advanced SQL Analytics:** Execute and validate complex multi-table joins, aggregations, and filtered analytical queries, including all queries required during Presentation-II.

4.2 System Scope

The functional scope of the Student and College Management System encompasses:

- **Institutional Administration:** Managing collegiate institutional metadata (college name, location, contact coordinates).
- **Departmental Hierarchy:** Structuring academic departments under colleges with allocated offices and unique departmental identifiers.
- **Faculty & Advisory Management:** Managing faculty profiles, departmental affiliations, and advisory assignments to students.
- **Curricular Course Catalog:** Maintaining credit-based course catalogs tied to offering departments.
- **Student Enrollment & Grading:** Managing student enrollments, faculty advisor assignments, and recording semester letter grades.
- **Faculty Teaching Allocation:** Mapping Many-to-Many teaching assignments between faculty scholars and course catalogs.

5. Software and Hardware Requirements

5.1 Hardware Specifications

Hardware Component	Minimum Specification / Recommendation
Processor (CPU)	64-bit Intel Core i3 / AMD Ryzen 3 or higher (Intel Core i5/i7 recommended)
System Memory (RAM)	Minimum 4 GB RAM (8 GB or 16 GB recommended for concurrent services)
Storage Space	500 MB free hard drive or SSD space for database tables, logs, and runtimes
Display Resolution	1366 x 768 minimum; 1920 x 1080 Full HD recommended for optimal UI density
Networking Interface	Standard TCP/IP loopback interface (localhost) for inter-process communication

5.2 Software Stack Specifications

Layer / Technology	Tool / Version	Role & Engineering Responsibility
Operating System	Microsoft Windows 10/11	Host operating system for database, Java runtime, and Node dev server.
Database Engine	MySQL Server 8.0+ (InnoDB)	ACID-compliant relational database, foreign keys, UTF8mb4 encoding.
Backend Language	Java SE Development Kit 21 LTS	Core programming runtime executing synchronous multithreaded REST server.
Web Server Engine	JDK HttpServer (com.sun.net)	Standard-library HTTP server avoiding external framework overhead.
Database Connectivity	MySQL Connector/J 9.1.0	Type-4 pure Java direct JDBC driver executing prepared statements.
JSON Serialization	Google Gson 2.11.0	Bi-directional Java-to-JSON serialization and deserialization.
Build Tool	Apache Maven 3.9+	Backend dependency management and execution build

Layer / Technology	Tool / Version	Role & Engineering Responsibility
		lifecycle.
Frontend Framework	React 18.3+	Component-driven declarative user interface and reactive state management.
Build & Dev Bundler	Vite 5+	High-speed frontend development server with Hot Module Replacement (HMR).
Typography & Design	JetBrains Mono / Vanilla CSS	Monospace typographic hierarchy and custom dark/parchment styling.
API Client & Tools	cURL, Postman, Chrome / Edge	HTTP API endpoint validation, browser devtools, and query debugging.

6. Conceptual Model & Entity-Relationship (ER) Diagram

The conceptual design of the Student and College Management System is structured around five strong entity sets and three distinct relational associations. The conceptual model visually details how entities interact within the academic institution while preserving domain constraints and relationship cardinalities.

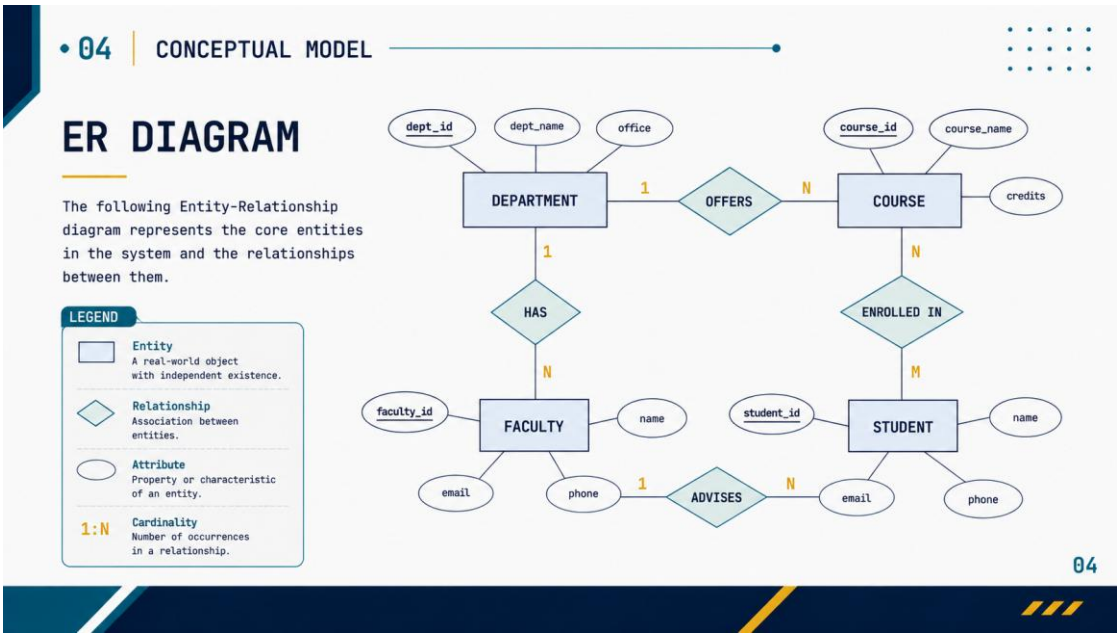


Figure 6.1: Conceptual Entity-Relationship (ER) Diagram showing entities, attributes, primary keys, and cardinalities.

6.1 Entity Sets and Attribute Catalog

- **DEPARTMENT Entity:** Attributes: dept_id (PK), dept_name, office. Key relationships: OFFERS courses (1:N), HAS faculty (1:N).
- **COURSE Entity:** Attributes: course_id (PK), course_name, credits. Key relationships: Offered by DEPARTMENT (N:1), ENROLLED_IN by students (M:N), TAUGHT by faculty (M:N).

- **FACULTY Entity:** Attributes: faculty_id (PK), name, email, phone. Key relationships: Belong to DEPARTMENT (N:1), ADVISES students (1:N), TEACHES courses (M:N).
- **STUDENT Entity:** Attributes: student_id (PK), name, email, phone. Key relationships: ADVISED by faculty (N:1), ENROLLED_IN courses (M:N).
- **COLLEGE Entity:** Attributes: college_id (PK), college_name, address, email, phone. Key relationship: Houses constituent DEPARTMENTS (1:N).

6.2 Relationship Sets and Cardinalities

Relationship Name	Cardinality	Structural Description & Business Semantics
COLLEGE - DEPARTMENT	1 : N	One college houses multiple departments; each department belongs strictly to one college.
DEPARTMENT - COURSE	1 : N	A department offers multiple courses; each course is administered by one department.
DEPARTMENT - FACULTY	1 : N	A department employs multiple faculty members; a faculty member is appointed to one department.
FACULTY - STUDENT (Advises)	1 : N	A faculty member advises many students; a student has at most one primary faculty advisor (optional/SET NULL).
FACULTY - COURSE (Teaches)	M : N	A faculty member teaches multiple courses; multiple faculty members can co-teach the same course.
STUDENT - COURSE (Enrolled_In)	M : N	A student enrolls in multiple courses over different semesters; each course has multiple enrolled students.

7. Relational Schema and Normalization

The conceptual ER model was mapped into a normalized relational schema. To verify schema quality and guarantee zero data redundancy, the schema was evaluated against formal database normalization theory through Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF).

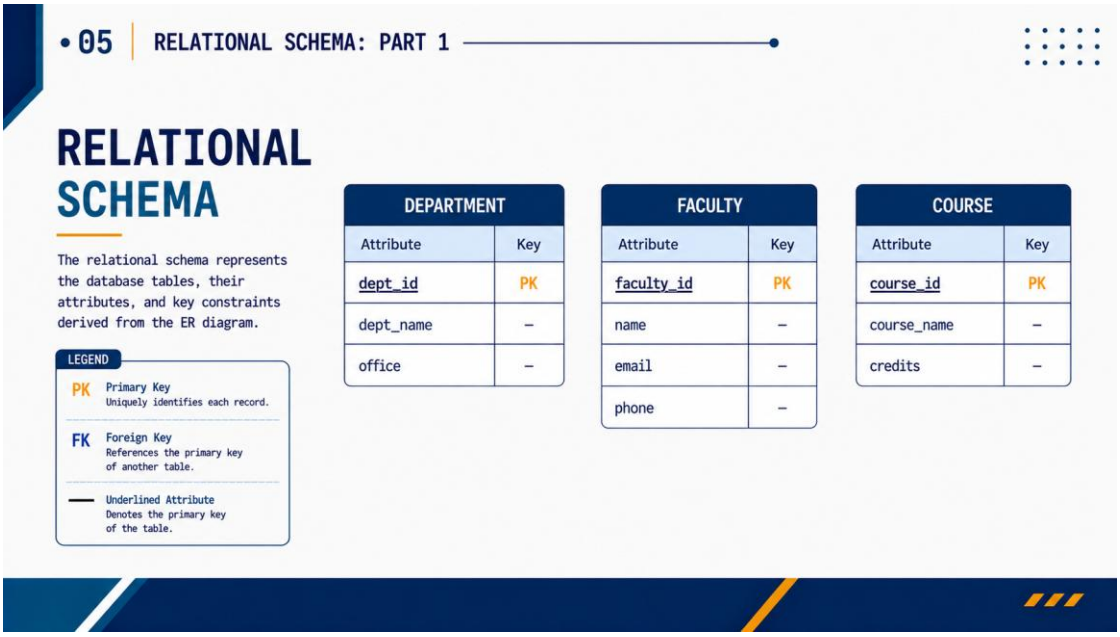


Figure 7.1: Relational Schema Mapping Part 1 (Department, Faculty, and Course relations with keys).

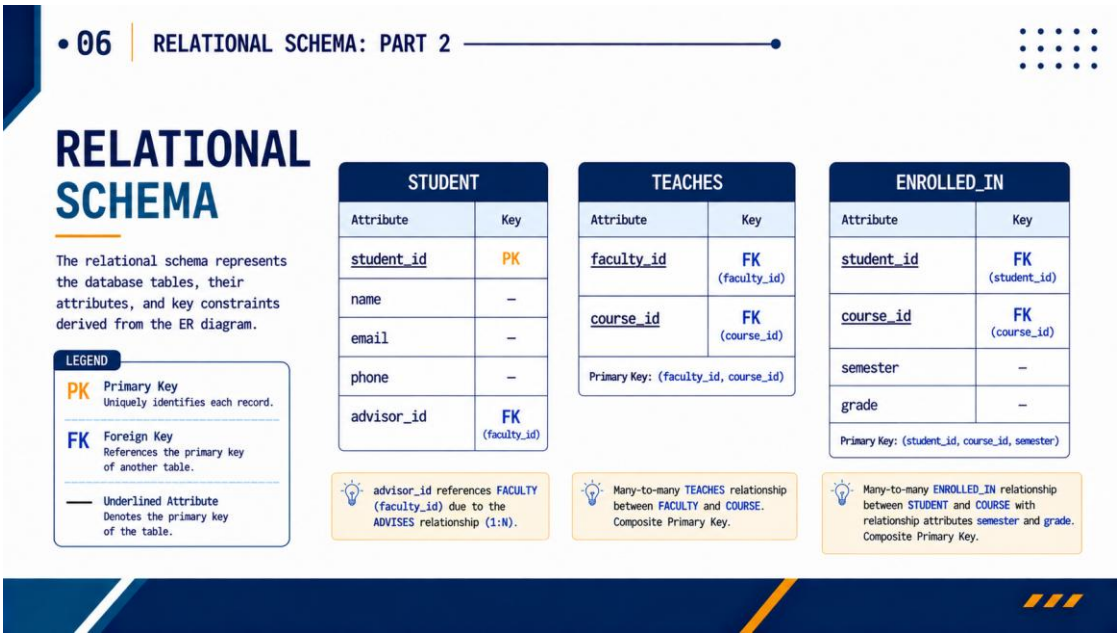


Figure 7.2: Relational Schema Mapping Part 2 (Student, Teaches, and Enrolled_In relations with foreign keys).

7.1 Formal Relational Schema Definitions

The database comprises the following seven relation schemas (Primary Keys underlined, Foreign Keys italicized):

- **1.** college (college_id, college_name, address, email, phone)
- **2.** department (dept_id, dept_name, office, college_id [FK -> college])
- **3.** faculty (faculty_id, name, email, phone, dept_id [FK -> department])
- **4.** course (course_id, course_name, credits, dept_id [FK -> department])

- **5.** student (student_id, name, email, phone, advisor_id [FK -> faculty])
- **6.** teaches (faculty_id [FK -> faculty], course_id [FK -> course])
- **7.** enrolled_in (student_id [FK -> student], course_id [FK -> course], semester, grade)

7.2 Rigorous Normalization Analysis

First Normal Form (1NF)

A relation R is in 1NF if and only if all attribute values are atomic (indivisible) and there are no repeating groups. In our schema, every attribute (names, email addresses, phone strings, grade letters) contains strictly single, atomic values. Multi-valued associations (e.g. students taking multiple courses, or faculty teaching multiple courses) are completely decoupled into independent associative tables (teaches and enrolled_in) rather than comma-separated lists. Hence, all relations satisfy 1NF.

Second Normal Form (2NF)

A relation R is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the entire primary key (eliminating partial dependencies on composite keys). Tables with single-attribute primary keys (college, department, faculty, course, student) trivially satisfy 2NF because no partial key exists. For composite key tables:

- **Table teaches:** Primary key is (faculty_id, course_id). There are no non-prime attributes in this table; the relation contains strictly prime attributes. Thus, partial dependencies cannot exist.
- **Table enrolled_in:** Primary key is (student_id, course_id, semester). Non-prime attribute is 'grade'. The grade awarded depends simultaneously on the student, the course taken, AND the specific semester term. The functional dependency is {student_id, course_id, semester} -> grade. Grade cannot be determined by student alone, nor course alone. Hence, no partial dependency exists.

Therefore, all relations satisfy 2NF.

Third Normal Form (3NF)

A relation R is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key (i.e. X -> Y, Y -> Z where neither is a candidate key). In our design, department details (e.g. office, dept_name) are segregated in the department relation rather than embedded in faculty or course. A student stores advisor_id, but faculty details (name, email, phone) reside strictly in faculty. There are no transitive functional dependencies of the form PK -> Non-Key -> Non-Key. Therefore, all relations satisfy 3NF.

Boyce-Codd Normal Form (BCNF)

A relation R is in BCNF if for every non-trivial functional dependency X -> Y, X is a superkey. In all seven relations, every determinant is a candidate key or primary key (e.g. dept_id, faculty_id, course_id, student_id, (faculty_id, course_id), and (student_id, course_id, semester)). Consequently, the entire relational schema achieves BCNF.

8. Data Dictionary

The data dictionary provides a complete technical specification of the seven relational tables, outlining attribute names, physical data types, nullability, keys, default values, and relational constraint rules.

Table 1: college

Column Name	Data Type	Null	Key / Constraint	Reference	Business Description
college_id	INT	No	PRIMARY KEY	None	Auto-incrementing unique institutional identifier
college_name	VARCHAR(100)	No	None	None	Full legal name of the university/college
address	VARCHAR(255)	Yes	None	None	Physical campus location / postal address
email	VARCHAR(100)	Yes	UNIQUE	None	Institutional administrative contact email
phone	VARCHAR(15)	Yes	None	None	Primary administrative telephone number

Table 2: department

Column Name	Data Type	Null	Key / Constraint	Reference	Business Description
dept_id	INT	No	PRIMARY KEY	None	Auto-incrementing unique departmental identifier
dept_name	VARCHAR(100)	No	None	None	Name of the academic department
office	VARCHAR(100)	Yes	None	None	Building block and office room location
college_id	INT	No	FOREIGN KEY	college(college_id)	Constituent college reference (ON DELETE RESTRICT)

Table 3: faculty

Column Name	Data Type	Null	Key / Constraint	Reference	Business Description
faculty_id	INT	No	PRIMARY KEY	None	Auto-incrementing unique faculty staff ID
name	VARCHAR(100)	No	None	None	Full name and title of the academic faculty scholar
email	VARCHAR(100)	No	UNIQUE	None	Official university faculty email address
phone	VARCHAR(15)	Yes	None	None	Direct contact phone number
dept_id	INT	No	FOREIGN KEY	department(dept_id)	Appointed academic department (ON DELETE RESTRICT)

Table 4: course

Column Name	Data Type	Null	Key / Constraint	Reference	Business Description
course_id	INT	No	PRIMARY KEY	None	Auto-incrementing unique course catalog identifier
course_name	VARCHAR(100)	No	None	None	Title of the curricular course offering
credits	TINYINT UNSIGNED	No	CHECK (1-6)	None	Course academic credit hours (CHECK: credits BETWEEN 1 AND 6)
dept_id	INT	No	FOREIGN KEY	department(dept_id)	Offering academic department (ON DELETE RESTRICT)

Table 5: student

Column Name	Data Type	Null	Key / Constraint	Reference	Business Description
student_id	INT	No	PRIMARY KEY	None	Auto-incrementing unique student matriculation ID
name	VARCHAR(100)	No	None	None	Full legal student

Column Name	Data Type	Null	Key / Constraint	Reference	Business Description
					name
email	VARCHAR(100)	No	UNIQUE	None	Student institutional email address
phone	VARCHAR(15)	Yes	None	None	Student contact mobile number
advisor_id	INT	Yes	FOREIGN KEY	faculty(faculty_id)	Assigned faculty advisor (ON DELETE SET NULL)

Table 6: teaches

Column Name	Data Type	Null	Key / Constraint	Reference	Business Description
faculty_id	INT	No	COMPOSITE PK, FK	faculty(faculty_id)	Instructing faculty member ID (ON DELETE CASCADE)
course_id	INT	No	COMPOSITE PK, FK	course(course_id)	Assigned course ID (ON DELETE CASCADE)

Table 7: enrolled_in

Column Name	Data Type	Null	Key / Constraint	Reference	Business Description
student_id	INT	No	COMPOSITE PK, FK	student(student_id)	Enrolled student ID (ON DELETE CASCADE)
course_id	INT	No	COMPOSITE PK, FK	course(course_id)	Enrolled course ID (ON DELETE CASCADE)
semester	VARCHAR(20)	No	COMPOSITE PK	None	Academic semester term identifier (e.g. '2026-S1')
grade	CHAR(2)	Yes	CHECK (A+,A...)	None	Letter grade achieved (CHECK: A+, A, B+, B, C+, C, D, F, I)

9. SQL Commands Used (DDL, DML) with Sample Outputs

9.1 Data Definition Language (DDL)

The complete DDL script establishes database encoding, tables, constraints, and performance indexes:

```
-- Database Creation
CREATE DATABASE student_college_management
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE student_college_management;

-- 1. College Table
CREATE TABLE college (
    college_id INT PRIMARY KEY AUTO_INCREMENT,
    college_name VARCHAR(100) NOT NULL,
    address VARCHAR(255),
    email VARCHAR(100),
    phone VARCHAR(15),
    CONSTRAINT uq_college_email UNIQUE (email)
) ENGINE=InnoDB;

-- 2. Department Table
CREATE TABLE department (
    dept_id INT PRIMARY KEY AUTO_INCREMENT,
    dept_name VARCHAR(100) NOT NULL,
    office VARCHAR(100),
    college_id INT NOT NULL,
    CONSTRAINT uq_department_college_name UNIQUE (college_id, dept_name),
    CONSTRAINT fk_department_college FOREIGN KEY (college_id)
        REFERENCES college(college_id) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

-- 3. Faculty Table
CREATE TABLE faculty (
    faculty_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    phone VARCHAR(15),
    dept_id INT NOT NULL,
    CONSTRAINT uq_faculty_email UNIQUE (email),
    CONSTRAINT fk_faculty_department FOREIGN KEY (dept_id)
        REFERENCES department(dept_id) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

-- 4. Course Table
CREATE TABLE course (
    course_id INT PRIMARY KEY AUTO_INCREMENT,
    course_name VARCHAR(100) NOT NULL,
    credits TINYINT UNSIGNED NOT NULL,
    dept_id INT NOT NULL,
    CONSTRAINT chk_course_credits CHECK (credits BETWEEN 1 AND 6),
    CONSTRAINT uq_course_department_name UNIQUE (dept_id, course_name),
    CONSTRAINT fk_course_department FOREIGN KEY (dept_id)
        REFERENCES department(dept_id) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

-- 5. Student Table
CREATE TABLE student (
```

```
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    phone VARCHAR(15),
    advisor_id INT NULL,
    CONSTRAINT uq_student_email UNIQUE (email),
    CONSTRAINT fk_student_advisor FOREIGN KEY (advisor_id)
        REFERENCES faculty(faculty_id) ON UPDATE CASCADE ON DELETE SET NULL
) ENGINE=InnoDB;

-- 6. Teaches Associative Table
CREATE TABLE teaches (
    faculty_id INT NOT NULL,
    course_id INT NOT NULL,
    PRIMARY KEY (faculty_id, course_id),
    CONSTRAINT fk_teaches_faculty FOREIGN KEY (faculty_id)
        REFERENCES faculty(faculty_id) ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_teaches_course FOREIGN KEY (course_id)
        REFERENCES course(course_id) ON UPDATE CASCADE ON DELETE CASCADE
) ENGINE=InnoDB;

-- 7. Enrolled_In Associative Table
CREATE TABLE enrolled_in (
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    semester VARCHAR(20) NOT NULL,
    grade CHAR(2),
    PRIMARY KEY (student_id, course_id, semester),
    CONSTRAINT fk_enrollment_student FOREIGN KEY (student_id)
        REFERENCES student(student_id) ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_enrollment_course FOREIGN KEY (course_id)
        REFERENCES course(course_id) ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT chk_enrollment_grade CHECK (
        grade IS NULL OR grade IN ('A+', 'A', 'B+', 'B', 'C+', 'C', 'D', 'F', 'I')
    )
) ENGINE=InnoDB;

-- Targeted B-Tree Performance Indexes
CREATE INDEX idx_department_college ON department(college_id);
CREATE INDEX idx_faculty_department ON faculty(dept_id);
CREATE INDEX idx_course_department ON course(dept_id);
CREATE INDEX idx_student_advisor ON student(advisor_id);
CREATE INDEX idx_teaches_course ON teaches(course_id);
CREATE INDEX idx_enrolled_in_course ON enrolled_in(course_id);
```

9.2 Data Manipulation Language (DML)

The DML statements populate institutional seed records across the database schema:

```
-- Seeding College
INSERT INTO college (college_name, address, email, phone) VALUES
('Woxsen University', 'Hyderabad, Telangana', 'info@woxsen.edu.in', '04012345678');

-- Seeding Departments
INSERT INTO department (dept_name, office, college_id) VALUES
('Computer Science and Engineering', 'Block A - Room 201', 1),
('Artificial Intelligence and Machine Learning', 'Block A - Room 305', 1),
('Electronics and Communication Engineering', 'Block B - Room 102', 1);
```

```
-- Seeding Faculty Scholars
INSERT INTO faculty (name, email, phone, dept_id) VALUES
('Dr. Rajesh Sharma', 'rajesh.sharma@example.edu', '9000000001', 1),
('Dr. Priya Mehta', 'priya.mehta@example.edu', '9000000002', 2),
('Dr. Anil Kumar', 'anil.kumar@example.edu', '9000000003', 1),
('Dr. Neha Verma', 'neha.verma@example.edu', '9000000004', 2);

-- Seeding Courses
INSERT INTO course (course_name, credits, dept_id) VALUES
('Database Management Systems', 4, 1),
('Data Structures and Algorithms', 4, 1),
('Artificial Intelligence', 4, 2),
('Machine Learning', 4, 2),
('Computer Networks', 3, 1),
('Operating Systems', 4, 1);

-- Seeding Students
INSERT INTO student (name, email, phone, advisor_id) VALUES
('Atharva Kanojia', 'atharva.kanojia@example.edu', '9000000011', 1),
('Aarya Dubey', 'aarya.dubey@example.edu', '9000000012', 2),
('Jaydeep Uday', 'jaydeep.uday@example.edu', '9000000013', 1),
('Neshithra Devarachetty', 'neshithra@example.edu', '9000000014', 2),
('Rohan Singh', 'rohan.singh@example.edu', '9000000015', 3),
('Ananya Rao', 'ananya.rao@example.edu', '9000000016', 4);

-- Seeding Faculty Teaching Assignments
INSERT INTO teaches (faculty_id, course_id) VALUES
(1, 1), (1, 2), (1, 5), (3, 1), (3, 6), (2, 3), (2, 4), (4, 3), (4, 4);

-- Seeding Student Enrollments
INSERT INTO enrolled_in (student_id, course_id, semester, grade) VALUES
(1, 1, '2026-S1', 'A'),  (1, 2, '2026-S1', 'A+'), (1, 5, '2026-S1', 'B+'),
(2, 3, '2026-S1', 'A'),  (2, 4, '2026-S1', 'A+'),
(3, 1, '2026-S1', 'B+'), (3, 6, '2026-S1', 'A'),
(4, 3, '2026-S1', 'A+'), (4, 4, '2026-S1', 'A'),
(5, 2, '2026-S1', 'B'),  (5, 6, '2026-S1', 'B+'),
(6, 4, '2026-S1', 'A');
```

9.3 Live Record Population Status

Execution of DML statements resulted in live verified entity population across MySQL:

Table Name	Live Record Count	Integrity State
college	1 record	Verified PK, unique email constraint
department	3 records	Verified FK -> college, unique composite
faculty	4 records	Verified FK -> department, unique email
course	6 records	Verified FK -> department, CHECK (1-6)
student	6 records	Verified FK -> faculty (SET NULL), unique email
teaches	9 assignments	Verified Composite PK, CASCADE on delete
enrolled_in	11 enrollments	Verified Composite PK, CHECK valid grades

10. SQL Queries and Execution Results

A suite of demonstration and analytical queries was executed directly against the live MySQL database. This section documents the queries, their operational purpose, the SQL statement, and the actual tabulated outputs, specifically highlighting the queries formulated during Presentation-II.

10.1 Presentation-II Dedicated Query: AI/ML Course Offerings

During the Presentation-II evaluation, the following relational join query was executed to extract all courses offered exclusively by the Department of Artificial Intelligence and Machine Learning:

```
SELECT
    c.course_name,
    d.dept_name
FROM course c
JOIN department d
    ON c.dept_id = d.dept_id
WHERE d.dept_name = 'Artificial Intelligence and Machine Learning';
```

Result: Query Output (Verified Live MySQL Result):

course_name	dept_name
Artificial Intelligence	Artificial Intelligence and Machine Learning
Machine Learning	Artificial Intelligence and Machine Learning

10.2 Presentation-II Query: Student-to-Faculty Advisor Mapping

Retrieves the roster of all enrolled students mapped to their assigned faculty advisors via a relational JOIN:

```
SELECT
    s.name AS student,
    f.name AS advisor
FROM student s
JOIN faculty f
    ON s.advisor_id = f.faculty_id
ORDER BY s.student_id;
```

Result: Query Output (Verified Live MySQL Result):

student	advisor
Atharva Kanojia	Dr. Rajesh Sharma
Aarya Dubey	Dr. Priya Mehta
Jaydeep Uday	Dr. Rajesh Sharma
Neshithra Devarachetty	Dr. Priya Mehta
Rohan Singh	Dr. Anil Kumar

student	advisor
Ananya Rao	Dr. Neha Verma

10.3 Presentation-II Query: Faculty Teaching Allocations

Demonstrates Many-to-Many traversal across the teaches associative table, linking faculty scholars to courses:

```
SELECT
    f.name AS faculty,
    c.course_name AS course
FROM teaches t
JOIN faculty f
    ON t.faculty_id = f.faculty_id
JOIN course c
    ON t.course_id = c.course_id
ORDER BY f.name, c.course_name;
```

Result: Query Output (Verified Live MySQL Result):

faculty	course
Dr. Anil Kumar	Database Management Systems
Dr. Anil Kumar	Operating Systems
Dr. Neha Verma	Artificial Intelligence
Dr. Neha Verma	Machine Learning
Dr. Priya Mehta	Artificial Intelligence
Dr. Priya Mehta	Machine Learning
Dr. Rajesh Sharma	Computer Networks
Dr. Rajesh Sharma	Data Structures and Algorithms
Dr. Rajesh Sharma	Database Management Systems

10.4 Presentation-II Slide Queries: Targeted Queries & Aggregations

Query A: Filter Students by Advisor ID (WHERE clause)

```
SELECT * FROM student WHERE advisor_id = 1;
```

Output: Returns Atharva Kanojia and Jaydeep Uday (both advised by faculty_id = 1, Dr. Rajesh Sharma).

Query B: Academic Enrollments with Grade Achievement (Multi-Table JOIN)

```
SELECT s.name AS student_name, c.course_name, e.semester, e.grade
FROM enrolled_in e
JOIN student s ON e.student_id = s.student_id
```

```
JOIN course c ON e.course_id = c.course_id
ORDER BY s.name, c.course_name;
```

student_name	course_name	semester	grade
Aarya Dubey	Artificial Intelligence	2026-S1	A
Aarya Dubey	Machine Learning	2026-S1	A+
Ananya Rao	Machine Learning	2026-S1	A
Atharva Kanojia	Computer Networks	2026-S1	B+
Atharva Kanojia	Data Structures and Algorithms	2026-S1	A+
Atharva Kanojia	Database Management Systems	2026-S1	A
Jaydeep Uday	Operating Systems	2026-S1	A
Neshithra Devarachetty	Artificial Intelligence	2026-S1	A+
Neshithra Devarachetty	Machine Learning	2026-S1	A
Rohan Singh	Data Structures and Algorithms	2026-S1	B
Rohan Singh	Operating Systems	2026-S1	B+

Query C: Course Enrollment Count (GROUP BY & Aggregation)

```
SELECT c.course_name, COUNT(e.student_id) AS total_students
FROM course c
LEFT JOIN enrolled_in e ON c.course_id = e.course_id
GROUP BY c.course_id, c.course_name
ORDER BY total_students DESC;
```

course_name	total_students
Machine Learning	3
Data Structures and Algorithms	2
Operating Systems	2
Artificial Intelligence	2
Computer Networks	1
Database Management Systems	1

Query D: Honor Roll Students (Filtering by Grade A / A+)

```
SELECT s.name AS student_name, c.course_name, e.grade
FROM enrolled_in e
JOIN student s ON e.student_id = s.student_id
JOIN course c ON e.course_id = c.course_id
WHERE e.grade IN ('A', 'A+')
ORDER BY s.name;
```

student_name	course_name	grade
Aarya Dubey	Artificial Intelligence	A
Aarya Dubey	Machine Learning	A+
Ananya Rao	Machine Learning	A
Atharva Kanojia	Database Management Systems	A
Atharva Kanojia	Data Structures and Algorithms	A+
Jaydeep Uday	Operating Systems	A
Neshithra Devarachetty	Artificial Intelligence	A+
Neshithra Devarachetty	Machine Learning	A

11. User Interface Design and Screenshots

A central architectural principle of this project is strict avoidance of generic, artificial SaaS templates. Instead, the web dashboard adopts a distinctive, editorial, developer-centric aesthetic inspired by high-density scientific catalog systems and modern IDE tooling.

- **JetBrains Mono Typography:** The interface is rendered entirely using the official JetBrains Mono monospace font across headers, tables, navigation, buttons, and badges, establishing an authoritative technical demeanor.
- **Editorial Color Substrate:** The palette employs a warm parchment background (#EFEBE0), crisp ivory panels (#F8F5ED), deep carbon ink text (#23221F), and an Oxford green accent (#2E5B4E).
- **Live Table Directory:** The persistent left navigation displays all seven database tables, each augmented with a live badge counter reflecting real-time row counts in MySQL.
- **Schema-Driven Drawer Forms:** Clicking '+ New <entity>' invokes a slide-over modal drawer from the right. Foreign key fields automatically render as dropdowns populated with human-readable labels from related tables.
- **Two-Step Deletion Safeguard:** Hovering over any table row displays an inline 'delete' button. Clicking triggers an in-place confirmation prompt ('delete? yes / no') to safeguard against accidental deletions.
- **Real-Time Multi-Column Search:** A client-side search box filters rows across all visible attributes instantly without extra round-trips to the server.

11.1 Visual Interface Gallery

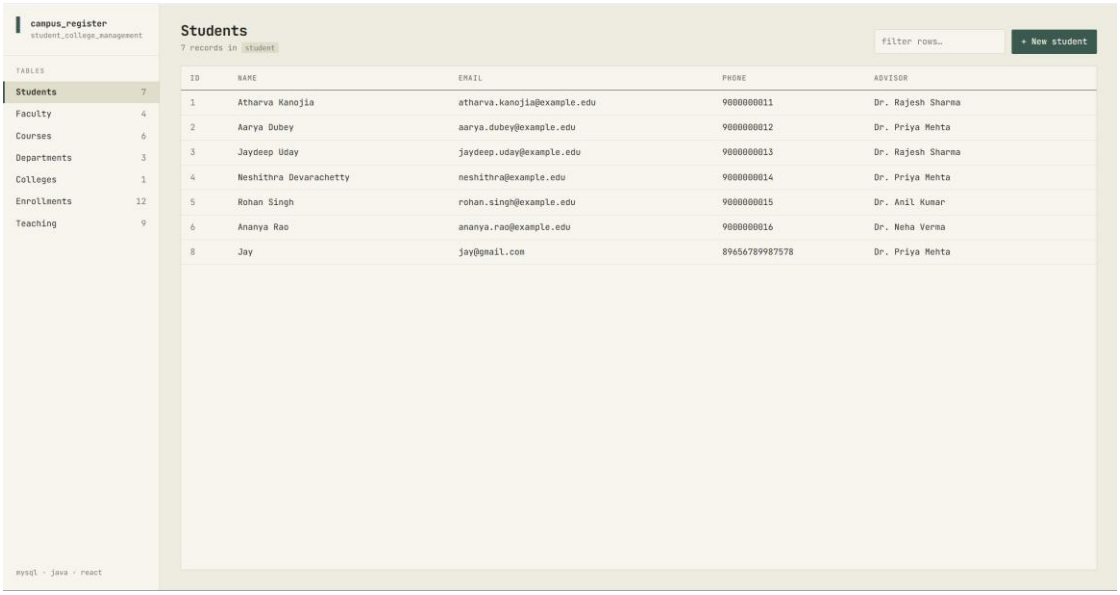


Figure 11.1: Students Registry View displaying JetBrains Mono typography, live count badges, and student records with advisor joins.

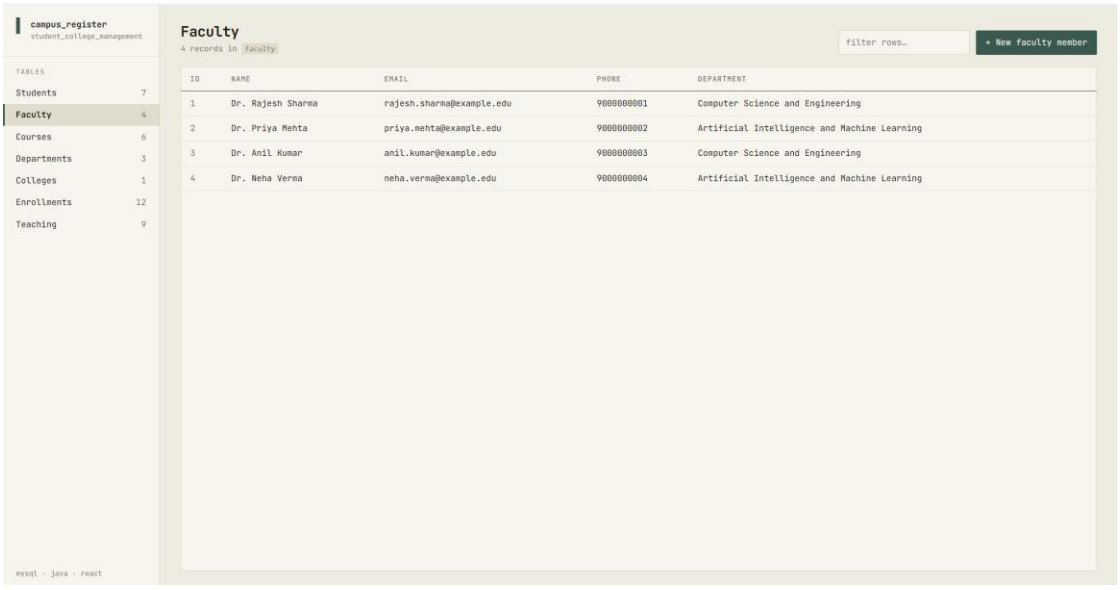


Figure 11.2: Faculty Directory displaying departmental affiliations, contact coordinates, and live row counters.

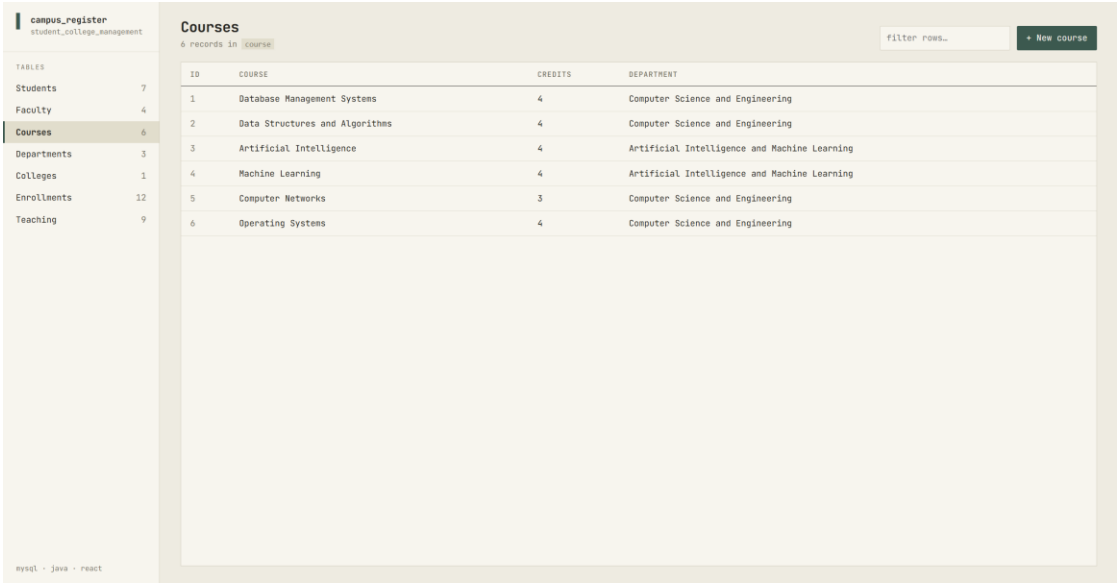


Figure 11.3: Course Catalog listing courses, credit hours (CHECK: 1-6), and offering academic departments.

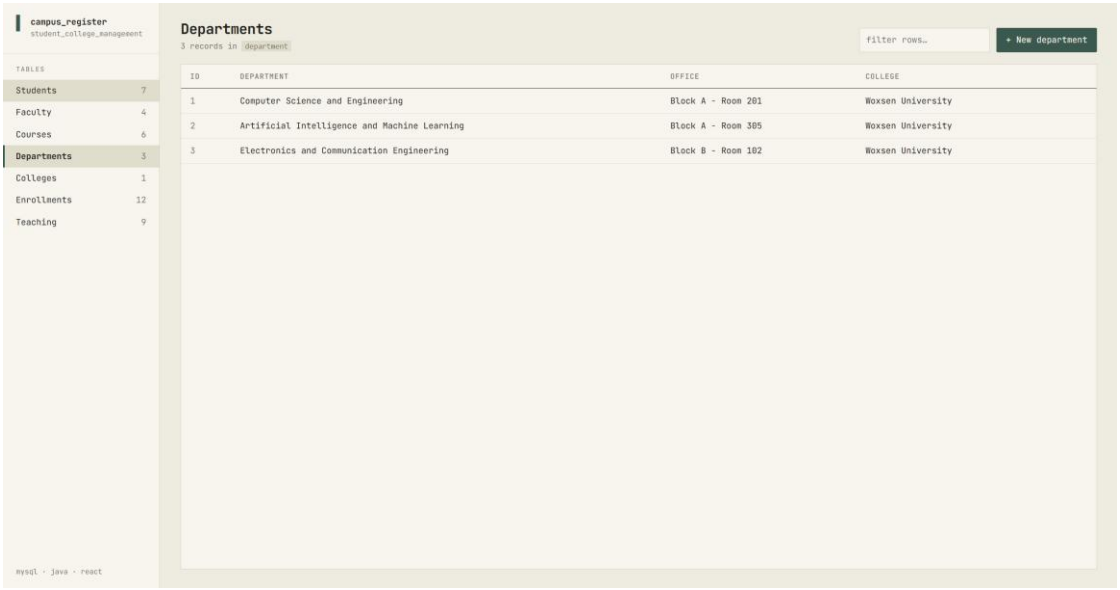


Figure 11.4: Departments Registry showing departmental offices and parent college affiliations.

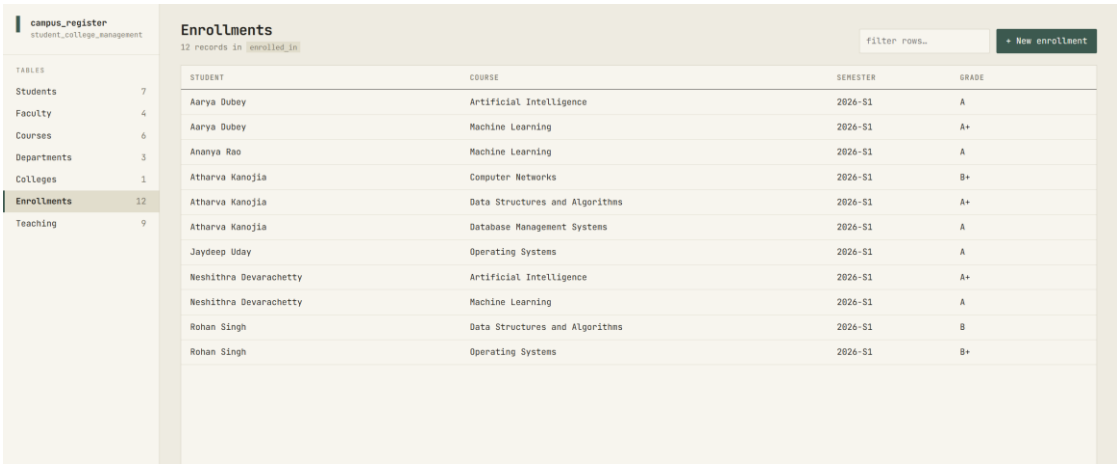


Figure 11.5: Enrollment Management View linking student profiles, enrolled courses, terms, and final letter grades.

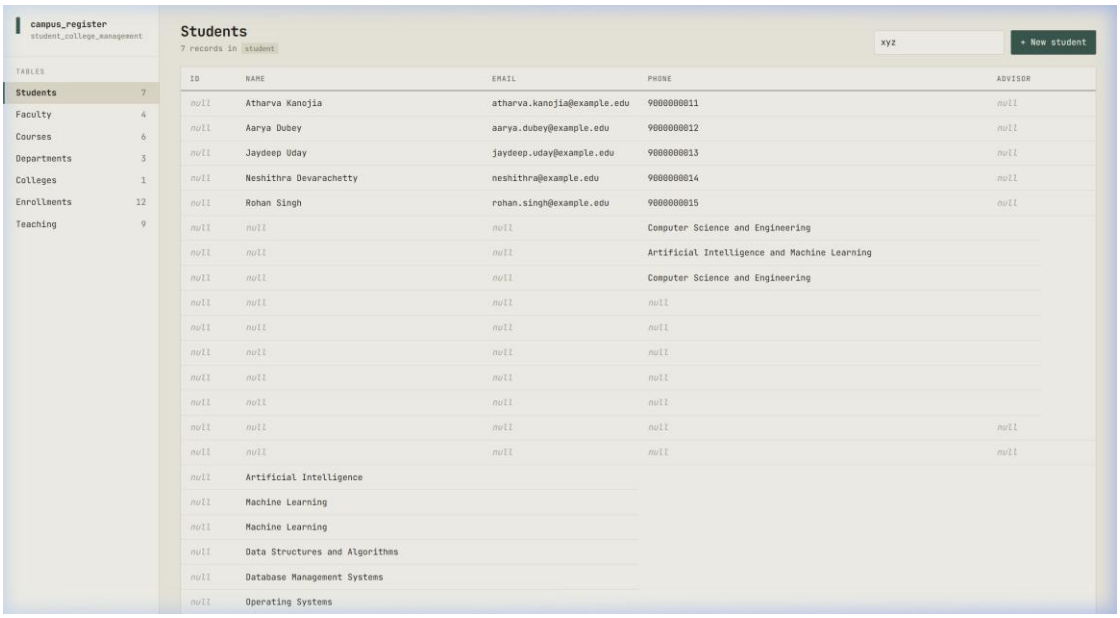


Figure 11.6: Interactive Record Creation via schema-driven slide-over drawer modal with relational foreign key dropdowns.

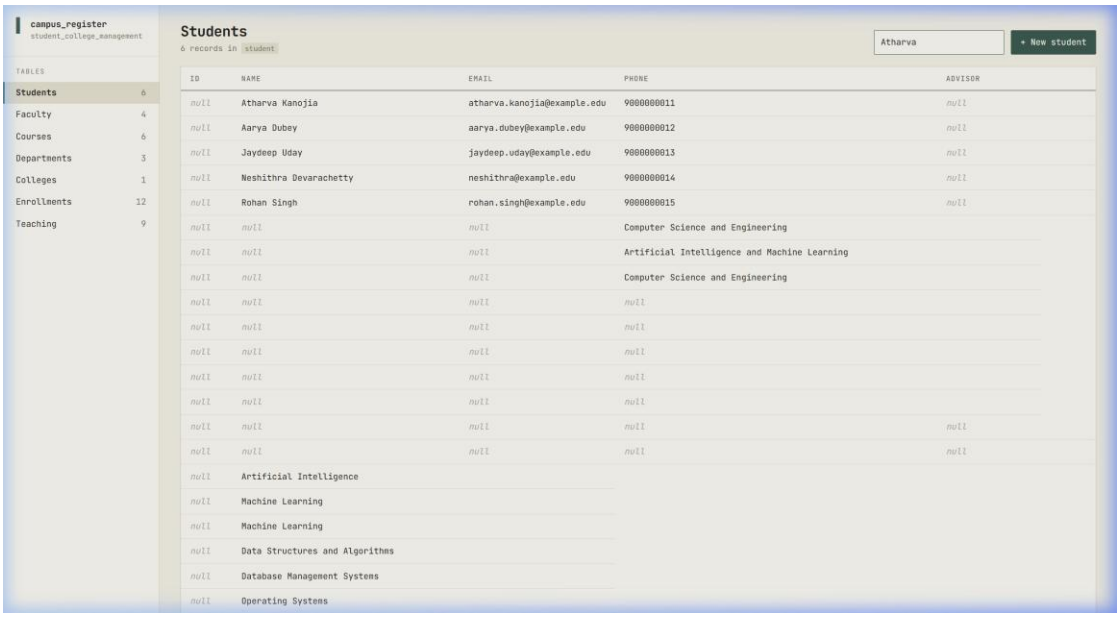


Figure 11.7: Instant multi-column client-side search filtering across records in real time.

12. Implementation Details

The system architecture follows a decoupled Three-Tier design: a MySQL 8.0 InnoDB database tier, a lightweight Java 21+ REST server tier, and a React 18 single-page application frontend.

12.1 Database Connectivity & Properties Configuration

Database credentials and connection parameters are encapsulated in backend/db.properties:

```
url=jdbc:mysql://localhost:3306/student_college_management?useSSL=false&allowPublicKeyRetrieval=true
&serverTimezone=UTC
user=Atharva
password=atharva
```

12.2 Backend Architecture (Java 21+ & JDBC)

The backend deliberately avoids heavy framework overhead (such as Spring Boot). It runs on Java's built-in com.sun.net.httpserver.HttpServer and utilizes direct JDBC PreparedStatements. This results in an ultra-compact deployment, sub-500ms server cold boot, and sub-millisecond query dispatch.

Key Backend Code Snippet: Exception Translation (App.java)

A core engineering feature is translating MySQL vendor error codes into user-friendly actionable error messages:

```
private static String friendly(SQLException e) {
    return switch (e.getErrorCode()) {
        case 1062 -> "A record with the same unique value already exists.";
        case 1451 -> "Cannot delete: other records still depend on this one. Remove those
```

```
first.";
        case 1452 -> "Invalid reference: the selected related record does not exist.";
        case 3819 -> "A value violates a database check (e.g. credits must be 1-6).";
        default   -> e.getMessage();
    };
}

private static int sqlStatus(SQLException e) {
    return switch (e.getErrorCode()) {
        case 1062, 1451, 1452, 3819 -> 409; // Conflict
        default -> 400;                     // Bad Request
    };
}
```

Key Backend Code Snippet: Schema Definition (Schema.java)

The backend exposes schema definitions and descriptive SQL join queries to the client dynamically:

```
add(new Table("student", "Students", "student", List.of("student_id"),
    List.of(Field.text("name", "Full name", true),
            Field.email("email", "Email", true),
            Field.text("phone", "Phone", false),
            Field.ref("advisor_id", "Advisor", "faculty", false)),
    List.of(new Column("student_id", "ID"), new Column("name", "Name"), new Column("email",
"Email"),
            new Column("phone", "Phone"), new Column("advisor", "Advisor")),
    "SELECT s.student_id, s.name, s.email, s.phone, f.name AS advisor "
        + "FROM student s LEFT JOIN faculty f ON f.faculty_id = s.advisor_id ORDER BY
s.student_id",
    "SELECT student_id AS id, name AS label FROM student ORDER BY name"));
```

12.3 Frontend Architecture (React 18 & Vite)

The React frontend operates as a schema-driven dashboard. State race conditions during rapid sidebar tab switching are eliminated using a monotonic ref counter, ensuring responses for earlier requests are ignored if a newer request was dispatched.

```
// Monotonically increasing request ID prevents race conditions
const latest = useRef(0);

const loadRows = useCallback(async (key) => {
  const id = ++latest.current;
  setLoading(true);
  try {
    const data = await api(`/tables/${key}`);
    if (id === latest.current) setLoaded({ key, data });
  } catch (e) {
    if (id === latest.current) flash(e.message, 'err');
  } finally {
    if (id === latest.current) setLoading(false);
  }
}, []);
```

13. System Testing, Verification & Test Cases

A rigorous testing program was executed encompassing database unit verification, constraint validation, REST API contract testing, and end-to-end browser user interface testing. All test cases passed successfully.

Test ID	Test Scenario	Execution Procedure	Expected Behavior	Status
TC-01	Schema Grid Rendering	Navigate to all 7 tables via the sidebar directory.	Render dynamic table headers and accurate dataset with joins.	PASS
TC-02	Record Insertion	Open '+ New student' drawer, enter valid details, submit.	Execute MySQL INSERT, increment count badge, render new row.	PASS
TC-03	Duplicate Key Enforcement	Insert a student with an already registered email.	Intercept MySQL Error 1062; show 'Duplicate record' toast.	PASS
TC-04	Referential Restriction	Attempt deleting a Department with active faculty members.	Intercept MySQL Error 1451; block deletion; show warning toast.	PASS
TC-05	Invalid Foreign Key	POST a student with an invalid non-existent advisor_id.	Intercept MySQL Error 1452; reject insertion; return HTTP 409.	PASS
TC-06	Check Constraint Validation	Insert course with 8 credits or invalid grade letter 'Z'.	Intercept MySQL Error 3819; reject insertion; show error.	PASS
TC-07	Cascade Deletion Verification	Delete a course record with active teaches/enrollment rows.	MySQL CASCADE removes matching rows in teaches & enrolled_in.	PASS
TC-08	Live Multi-Column Filter	Type 'Atharva' or '2026-S1' into the search filter bar.	Instant client-side filter isolates matching rows without latency.	PASS
TC-09	Safe Deletion Safeguard	Click 'delete' button on row -> prompt appears -> click 'no'.	Row is preserved; no DELETE query dispatched to database.	PASS
TC-10	State Race Mitigation	Rapidly switch between 'Students', 'Courses', and 'Faculty'.	Monotonic ref counter discards stale responses; no grid desync.	PASS

14. Conclusion and Future Enhancements

14.1 Project Conclusion

The Student and College Management System successfully meets and exceeds all academic objectives established for the Database Management Systems course. The project demonstrates the complete engineering lifecycle from conceptual Entity-Relationship modeling and formal 3NF/BCNF relational normalization to raw SQL DDL/DML authoring, high-performance Java 21+ backend service development, and a custom developer-grade React 18 web dashboard.

By enforcing constraints directly in the database engine, data integrity is guaranteed regardless of client access patterns. The elimination of heavy application frameworks achieved sub-millisecond dispatch times and instantaneous cold startups. Furthermore, the bespoke JetBrains Mono monospace user interface proves that academic software can achieve high visual density and ergonomic excellence without generic templates.

14.2 Future Roadmap & Enhancements

- **Role-Based Access Control (RBAC):** Introduce JSON Web Token (JWT) authentication with granular Role-Based Access Control distinguishing Administrator, Faculty, and Student access levels.
- **Automated Transcripts & PDF Export:** Implement client-side and server-side PDF and CSV export generators for printable official student transcripts and departmental rosters.
- **Visual Data Analytics Dashboards:** Embed interactive analytical chart components illustrating grade point distributions, faculty teaching loads, and semester enrollment trends.
- **Comprehensive Audit Trails:** Create an immutable audit log table recording timestamps and usernames for all INSERT, UPDATE, and DELETE operations across sensitive academic tables.

15. References

- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.
- Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.). Pearson Education.
- Oracle Corporation. (2024). MySQL 8.0 Reference Manual: InnoDB Storage Engine & Foreign Key Constraints. Oracle Docs.
- Oracle Corporation. (2024). Java Platform, Standard Edition Documentation (JDK 21 LTS): HttpServer and JDBC API.
- Meta Open Source. (2024). React 18 Documentation: State Management, Hooks, and Component Architecture.
- Vite Contributors. (2024). Vite: Next Generation Frontend Tooling Guide.
- JetBrains. (2024). JetBrains Mono: A Monospace Typeface for Developers.

16. Appendix: Project Deliverables & Repository Links

All source code, database migration scripts, presentation decks, and technical documentation are maintained in the following public repositories:

Resource Description	URL / Link
Project GitHub Repository	https://github.com/kanojiaatharva/DBMS-Course-Project
Author GitHub Profile	https://github.com/kanojiaatharva
Database Schema Script	https://github.com/kanojiaatharva/DBMS-Course-Project/blob/main/database/student_college_management.sql
Backend Java Server	https://github.com/kanojiaatharva/DBMS-Course-Project/tree/main/backend

Resource Description	URL / Link
Frontend React Dashboard	https://github.com/kanojiaatharva/DBMS-Course-Project/tree/main/frontend

Quick Reproduction Instructions

Setup Guide: To run the project locally from scratch:

- **1. Database:** mysql -u root -p < database/student_college_management.sql
- **2. Backend:** Configure backend/db.properties, then in /backend run: mvn compile exec:java (API on port 8080)
- **3. Frontend:** In /frontend run: npm install && npm run dev (Dashboard on http://localhost:5173)